

Week 06: Swarm Intelligence: Particles, Ants and Bees

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-06/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-06/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-06/lab.html>

Colab notebook with Python step 6 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-06/NB06_swarm_intelligence.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Flocks, ant colonies and bee hives solve problems that no individual could solve alone, using simple local rules and indirect communication. Swarm intelligence turns these mechanisms into optimisation algorithms [1]. Particle swarm optimisation moves candidate solutions through a continuous space under the pull of their own best and the swarm's best positions [2], ant colony optimisation builds tours with pheromone trails [3, 4], and the artificial bee colony algorithm divides the search among employed, onlooker and scout bees [5]. This week tunes a PID speed controller of a DC motor with a particle swarm, plans a drilling sequence for a circuit board with ants, and asks how to judge the many nature-inspired algorithms critically [6].

Estimated study time: 9 to 11 hours.

Midterm project. The midterm project is released this week and is due at the end of Week 7. It applies search, knowledge-based and fuzzy systems, and evolutionary and swarm optimisation to a problem of the student's department, and it counts 40 percent of the course grade. The [project page](#) describes the three options, the starter notebooks, the deliverables and the evaluation criteria.

Learning outcomes

By the end of the week, students are expected to explain the velocity and position update of particle swarm optimisation and the role of inertia, cognitive and social coefficients, to implement a particle swarm as a Python class, to formulate controller tuning as an optimisation problem with a time-domain cost, to explain how pheromone deposit and evaporation guide ant colony optimisation, to apply it to a sequencing problem, and to evaluate claims about new metaheuristics with sound experimental practice.

Python in this week

These notes close with Python step 6: Classes, objects and probabilistic choice. It covers classes, objects, attributes and methods for a DC motor model and the particles of a swarm, and the probabilistic choice and pheromone update of ant colony optimisation with NumPy [3, 7]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Collective behaviour as computation

A flock of birds turns as one body although no bird leads it. Reynolds reproduced this with three local rules for each simulated bird: avoid crowding neighbours, align with their heading, and move towards their centre [8]. Ant colonies find short paths between nest and food without a map, because ants deposit pheromone and prefer trails with more of it; shorter trails are completed faster and accumulate pheromone sooner. Honeybee colonies allocate foragers to flower patches in proportion to their quality through the waggle dance. These systems share decentralised control, simple agents, local interaction and emergent global behaviour, which Bonabeau, Dorigo and Theraulaz summarised as swarm intelligence [1].

Animation: Three rules make a flock. Each arrow follows only its neighbours. Set separation, alignment and cohesion and watch order emerge or dissolve. With alignment at zero, the flock never forms a common heading. Run it on the [lecture page](#).

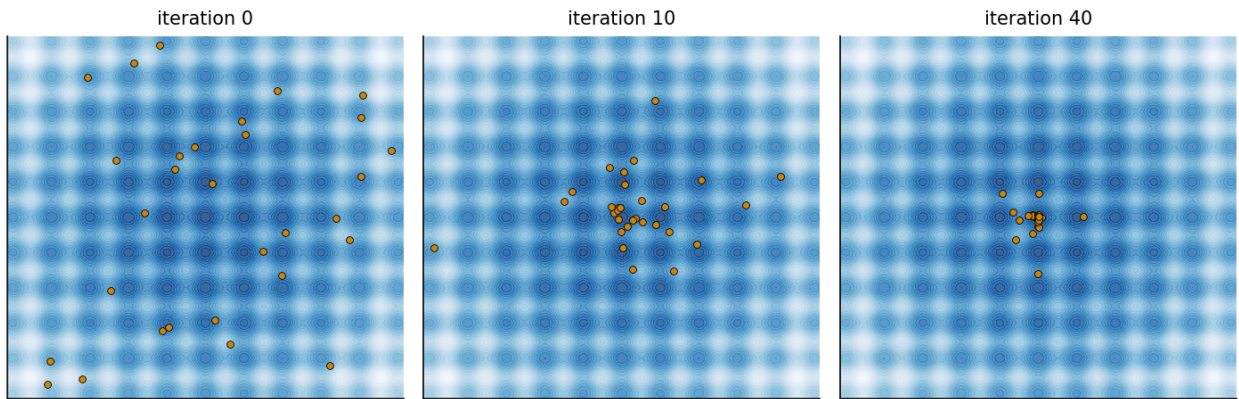


Figure 6.1. Particle swarm optimisation on a landscape with many local minima: positions of the swarm after 0, 10 and 40 iterations. The swarm first explores and then contracts around the best region [2].

Formulas: The three rules of a flock

Each simulated bird i has a position x_i and a velocity v_i , both vectors in the plane. Its neighbours N_i are the birds within a radius R , and $|N_i|$ is their number. Reynolds' three rules become three steering vectors [8]. Cohesion points from the bird to the centre of its neighbours, alignment is the difference between their mean velocity and its own, and separation points away from every neighbour closer than a smaller radius R_s :

$$c_i = \frac{1}{|N_i|} \sum_{j \in N_i} x_j - x_i, a_i = \frac{1}{|N_i|} \sum_{j \in N_i} v_j - v_i, s_i = - \sum_{j \in N_i, d_{ij} < R_s} \frac{x_j - x_i}{d_{ij}}$$

Here $d_{ij} = |x_j - x_i|$ is the distance between the two birds, so each separation term is a vector of length one. The velocity changes by a weighted sum of the three vectors, and the bird then moves by its velocity:

$$v_i \leftarrow v_i + k_s w_s s_i + k_a w_a a_i + k_c w_c c_i, x_i \leftarrow x_i + v_i$$

The weights w_s , w_a and w_c are the sliders of the animation, and the constants $k_s=0.0006$, $k_a=0.05$ and $k_c=0.004$ scale them to the units of the animation, in which the width of the field is 1. The animation also keeps every speed between 0.002 and 0.008 per step.

Worked example: One step of one bird

A bird at $x=(0.5, 0.3)$ flies with $v=(0.004, 0)$. The neighbourhood radius is $R=0.09$ and the separation radius $R_s=0.03$. Two other birds are near: bird A at $(0.52, 0.31)$ with velocity $(0.003, 0.002)$, and bird B at $(0.56, 0.34)$ with velocity $(0.006, 0.001)$. The weights are $w_s=1.5$, $w_a=1$ and $w_c=0.8$.

Step 1. Distances.

$$d_A = \sqrt{(0.52 - 0.5)^2 + (0.31 - 0.3)^2} = \sqrt{0.0004 + 0.0001} = 0.02236$$

$$d_B = \sqrt{(0.56 - 0.5)^2 + (0.34 - 0.3)^2} = \sqrt{0.0036 + 0.0016} = \sqrt{0.0052} = 0.07211$$

Both birds are neighbours, because both distances are below 0.09, but only A is closer than 0.03.

Step 2. Cohesion. The centre of the neighbours is $((0.52+0.56)/2, (0.31+0.34)/2) = (0.54, 0.325)$, so $c = (0.54, 0.325) - (0.5, 0.3) = (0.04, 0.025)$.

Step 3. Alignment. The mean velocity is $((0.003+0.006)/2, (0.002+0.001)/2) = (0.0045, 0.0015)$, so $a = (0.0045, 0.0015) - (0.004, 0) = (0.0005, 0.0015)$.

Step 4. Separation. Only A counts: $s = -\frac{(0.02, 0.01)}{0.02236} = (-0.8945, -0.4472)$.

Step 5. The new velocity and position. The three factors are $k_s w_s = 0.0006 \times 1.5 = 0.0009$, $k_a w_a = 0.05 \times 1 = 0.05$ and $k_c w_c = 0.004 \times 0.8 = 0.0032$:

$$\Delta v = 0.0009 \times (-0.8945, -0.4472) + 0.05 \times (0.0005, 0.0015) + 0.0032 \times (0.04, 0.025)$$

$$\Delta v = (-0.000805, -0.000402) + (0.000025, 0.000075) + (0.000128, 0.00008)$$

$$\Delta v_1 = -0.000805 + 0.000025 + 0.000128 = -0.000652$$

$$\Delta v_2 = -0.000402 + 0.000075 + 0.00008 = -0.000247$$

$$v = (0.004, 0) + (-0.000652, -0.000247) = (0.003348, -0.000247)$$

$$x = (0.5, 0.3) + (0.003348, -0.000247) = (0.503348, 0.299753)$$

The speed $\sqrt{0.003348^2 + (-0.000247)^2} = 0.003357$ lies inside the allowed range. The bird turns away from A, which is too close, and slightly towards the group, which is the behaviour that the animation shows for the whole flock.

Particle swarm optimisation

Kennedy and Eberhart introduced particle swarm optimisation in 1995, inspired by bird flocking and social behaviour [2]. Each particle i has a position x , which is a candidate solution, and a velocity v . It remembers the best position p it has visited, and the swarm shares the best position g found by any particle. In every iteration, the velocity is updated as $v = w v + c_1 r_1 (p - x) + c_2 r_2 (g - x)$, and the position as $x = x + v$, where r_1 and r_2 are random numbers between 0 and 1 drawn for each dimension.

The three terms have clear meanings. The inertia weight w keeps part of the previous motion; Shi and Eberhart introduced it to balance global and local search, with values around 0.4 to 0.9 [9]. The cognitive term, weighted by c_1 , pulls a particle back towards its own best experience. The social term, weighted by

c_2 , pulls it towards the best experience of the swarm. Clerc and Kennedy analysed the dynamics and derived a constriction factor that guarantees convergence; the widely used setting $w = 0.7298$ with $c_1 = c_2 = 1.49618$ follows from it [10]. Particle swarms are easy to implement, need no gradients, and work well on continuous problems of moderate dimension, but like every metaheuristic they can stagnate in a local optimum.

Check your understanding. In the velocity update, what does the social term $c_2 r_2 (g - x)$ do?

- A. It slows every particle down
- B. It pulls the particle towards the best position found by the whole swarm
- C. It pulls the particle towards its own best position
- D. It adds random noise independent of any position

Answer: B. g is the global best position shared by the swarm, so this term transmits the swarm's experience to each particle.

Formulas: The particle swarm update

Particle i has a position x_i and a velocity v_i . It remembers its personal best position p_i , and the swarm shares the global best position g . In every iteration and for every dimension d , two new random numbers r_1 and r_2 between 0 and 1 are drawn, and the particle moves by [2, 9]

$$v_{i,d} \leftarrow w v_{i,d} + c_1 r_1 (p_{i,d} - x_{i,d}) + c_2 r_2 (g_d - x_{i,d})$$

$$x_{i,d} \leftarrow x_{i,d} + v_{i,d}$$

The first term is the inertia, the second the cognitive pull towards the particle's own best position, and the third the social pull towards the best position of the swarm. After the move, the particle evaluates its new position and replaces p_i , and possibly g , if the new position is better. Clerc and Kennedy showed that the swarm converges when the velocity is multiplied by the constriction factor [10]

$$\chi = \frac{2}{|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|}, \varphi = \varphi_1 + \varphi_2 > 4$$

which gives the update $v \leftarrow \chi (v + \varphi_1 r_1 (p - x) + \varphi_2 r_2 (g - x))$. This is the inertia form above with $w = \chi$ and $c_1 = \chi \varphi_1$, $c_2 = \chi \varphi_2$.

Worked example: The constriction factor

With $\varphi_1 = \varphi_2 = 2.05$, so that $\varphi = 4.1$:

$$\varphi^2 - 4\varphi = 16.81 - 16.4 = 0.41, \sqrt{0.41} = 0.640312$$

$$\chi = \frac{2}{|2 - 4.1 - 0.640312|} = \frac{2}{2.740312} = 0.729844, c_1 = c_2 = 0.729844 \times 2.05 = 1.49618$$

These are the values $w = 0.7298$ and $c_1 = c_2 = 1.49618$ quoted in the lecture and used by the lab.

Worked example: One move of one particle

A particle searches the minimum of $f(x) = x_1^2 + x_2^2$. It is at $x = (2, -1)$ with $f = 4 + 1 = 5$ and velocity $v = (0.5, 0.3)$. Its best position is $p = (1.5, -0.5)$ with $f(p) = 2.25 + 0.25 = 2.5$, and the swarm's best is $g = (0.2, 0.1)$ with $f(g) = 0.04 + 0.01 = 0.05$. The random numbers are $r_1 = (0.6, 0.2)$ and $r_2 = (0.3, 0.9)$ for the two dimensions.

Step 1. First dimension.

$$v_1 = 0.7298 \times 0.5 + 1.49618 \times 0.6 \times (1.5 - 2) + 1.49618 \times 0.3 \times (0.2 - 2)$$

$$v_1 = 0.3649 + (-0.448854) + (-0.807937) = -0.891891, x_1 = 2 + (-0.891891) = 1.108109$$

Step 2. Second dimension.

$$v_2 = 0.7298 \times 0.3 + 1.49618 \times 0.2 \times ((-0.5) - (-1)) + 1.49618 \times 0.9 \times (0.1 - (-1))$$

$$v_2 = 0.21894 + 0.149618 + 1.481218 = 1.849776, x_2 = -1 + 1.849776 = 0.849776$$

Step 3. Evaluation and memory.

$$f(x) = 1.108109^2 + 0.849776^2 = 1.227906 + 0.722119 = 1.950025$$

The new value is better than the personal best 2.5, so p becomes (1.108109, 0.849776). It is worse than the global best 0.05, so g stays. In the first dimension the social term dominates and pulls the particle towards g ; in the second dimension both pulls point upwards, and the particle overshoots g , which is the oscillating search that the inertia and constriction keep under control. Part D of the [interactive lab](#) decomposes every update of a small swarm in the same way.

Tuning a PID controller with a swarm

The proportional-integral-derivative controller is the workhorse of industrial control. Its three gains shape the response: The proportional gain speeds up the reaction, the integral gain removes steady-state error, and the derivative gain damps oscillation [11]. Tuning them by hand is tedious, and classical rules assume simple process models. Controller tuning can instead be formulated as optimisation: Choose the gains that minimise a cost computed from a simulated step response, such as the integral of the time-weighted absolute error (ITAE), with penalties for overshoot and limits on the actuator. Particle swarms have been used in this way for automatic voltage regulators and many other loops [12].

The notebook applies the method to the speed control of a DC motor, using the standard model and parameters of the Control Tutorials for MATLAB and Simulink [13]. The armature voltage is limited to 24 volts, which makes the problem nonlinear, and the swarm evaluates all particles in one vectorised simulation. The cost function is part of the design: A cost that rewards speed alone drives the gains to their bounds, and adding a penalty on overshoot or control effort changes the optimum. The optimiser serves the engineer's definition of good behaviour; it cannot supply that definition.

Check your understanding. A swarm tunes a PID controller by minimising only the settling time, and the result shows violent oscillation of the actuator. What is the most likely cause?

- A. The swarm has too many particles
- B. The cost function does not penalise control effort or oscillation, so the optimiser exploits this
- C. PID controllers cannot be tuned by optimisation
- D. The inertia weight is exactly 0.7298

Answer: B. An optimiser minimises exactly what the cost states; unpenalised behaviour is free for it to exploit.

Formulas: PID control, the DC motor and the cost of a response

A PID controller computes its output u from the error $e = r - y$ between the set point r and the measured output y [11]:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de}{dt}$$

The integral adds up past errors and removes a steady offset; the derivative reacts to the rate of change and damps the motion. A digital controller with the sampling time Δt replaces the integral by a running sum and the derivative by a difference:

$$I_k = I_{k-1} + e_k \Delta t, D_k = \frac{e_k - e_{k-1}}{\Delta t}, u_k = K_p e_k + K_i I_k + K_d D_k$$

The lab filters the derivative with a time constant $\tau = 0.01$ s, $D_k^f = D_{k-1}^f + \frac{\Delta t}{\tau} (D_k - D_{k-1}^f)$, and limits the armature voltage to ± 24 V. The DC motor of the Control Tutorials couples the speed ω and the armature current i [13]:

$$J \frac{d\omega}{dt} = K i - b \omega, L \frac{di}{dt} = V - R i - K \omega$$

with inertia $J = 0.01$ kg m², friction $b = 0.1$ N m s, motor constant $K = 0.01$, resistance $R = 1$ Ω and inductance $L = 0.5$ H. At a constant voltage both derivatives vanish, so $i = b \omega / K$ and $V = R b \omega / K + K \omega$, which gives the steady speed

$$\omega_{ss} = \frac{K V}{R b + K^2}$$

The swarm judges a set of gains by the integral of the time-weighted absolute error over the simulated step response, approximated by a sum over the samples:

$$\text{ITAE} = \int_0^T t |e(t)| dt \approx \sum_k t_k |e_k| \Delta t$$

Late errors weigh more than early ones, so slow settling and lasting oscillation are expensive [12].

Worked example: The motor without control and the first two samples under PID control

Step 1. Open loop. One volt gives

$$\omega_{ss} = \frac{0.01 \times 1}{1 \times 0.1 + 0.01^2} = \frac{0.01}{0.1001} = 0.0999 \text{ rad/s}$$

so a constant voltage of $1/0.0999 = 10.01$ V would be needed for 1 rad/s, and any change of load or friction would shift the speed. Feedback removes this dependence.

Step 2. The first sample, $k = 0$. The controller of the lab uses $K_p = 100$, $K_i = 200$ and $K_d = 10$ with $\Delta t = 0.002$ s and a set point of 1 rad/s. The motor is at rest, so $e_0 = 1 - 0 = 1$. The integral is $I_0 = 0 + 1 \times 0.002 = 0.002$; the previous error is initialised to 1, so $D_0 = (1 - 1)/0.002 = 0$ and the filtered derivative stays 0. The controller asks for

$$u_0 = 100 \times 1 + 200 \times 0.002 + 10 \times 0 = 100 + 0.4 + 0 = 100.4 \text{ V}$$

which the limit cuts to $V = 24$ V. The motor equations give, with $\omega = 0$ and $i = 0$,

$$\frac{d\omega}{dt} = \frac{0.01 \times 0 - 0.1 \times 0}{0.01} = 0, \frac{di}{dt} = \frac{24 - 1 \times 0 - 0.01 \times 0}{0.5} = 48$$

and one Euler step of 0.002 s gives $\omega = 0$ and $i = 0 + 48 \times 0.002 = 0.096$ A.

Step 3. The second sample, $k = 1$. The speed is still 0, so $e_1 = 1$, $I_1 = 0.002 + 0.002 = 0.004$ and $D_1 = 0$. The demand $u_1 = 100 + 200 \times 0.004 = 100.8$ V is again cut to 24 V. Now the current drives the rotor:

$$\frac{d\omega}{dt} = \frac{0.01 \times 0.096 - 0.1 \times 0}{0.01} = 0.096, \frac{di}{dt} = \frac{24 - 1 \times 0.096 - 0.01 \times 0}{0.5} = 47.808$$

$$\omega = 0 + 0.096 \times 0.002 = 0.000192 \text{ rad/s}, i = 0.096 + 47.808 \times 0.002 = 0.191616 \text{ A}$$

Step 4. The cost so far. The ITAE sum receives $0 \times 1 \times 0.002 = 0$ from the first sample and $0.002 \times 1 \times 0.002 = 0.000004$ from the second. The voltage stays at its limit for many samples: The controller is saturated, which is the nonlinearity mentioned in the lecture. Part F of the [interactive lab](#) continues this table sample by sample for any gains, and Part B lets the swarm choose the gains.

Ant colony optimisation

Dorigo, Maniezzo and Colnari turned the pheromone mechanism into the Ant System for the travelling salesman problem [3]. Each artificial ant builds a complete tour city by city. From city i , it chooses the next unvisited city j with a probability proportional to the pheromone on edge (i, j) raised to a power α , times a heuristic desirability, usually the inverse distance, raised to a power β . After all ants have finished, pheromone evaporates on every edge by a factor ρ , and each ant deposits pheromone on the edges of its tour in inverse proportion to the tour length. Short tours therefore reinforce their edges, while evaporation forgets poor early choices.

The Ant Colony System added a stronger exploitation rule and local pheromone updates and was competitive on benchmark instances [14]; the monograph by Dorigo and Stützle presents the family and its applications to routing, scheduling and assignment [4]. Drilling holes in a printed circuit board is a classical travelling salesman application: The drill head must visit every hole once, and the travel time between holes is wasted production time. Benchmark libraries such as TSPLIB include instances that come from drilling problems [15].

Check your understanding. What is the purpose of pheromone evaporation in ant colony optimisation?

- A. It makes all tours equally likely forever
- B. It gradually removes pheromone from edges that are not reinforced, so poor early choices are forgotten
- C. It increases the length of short tours
- D. It fixes the start city

Answer: B. Without evaporation, early random choices would accumulate pheromone and lock the colony into poor tours.

Formulas: Ant System and Ant Colony System

An ant at city i chooses the next city j among the cities N_i it has not visited yet with the probability [3]

$$p_{ij} = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in N_i} \tau_{il}^\alpha \eta_{il}^\beta}$$

where τ_{ij} is the pheromone on the edge from i to j , $\eta_{ij} = 1/d_{ij}$ is the heuristic desirability, the inverse of the distance, and the exponents α and β set the weight of the two. When all m ants have built their tours of lengths L_k , the pheromone evaporates with the rate ρ (rho) and every ant deposits pheromone on the edges of its tour:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^k, \Delta \tau_{ij}^k = \frac{Q}{L_k} \text{ if ant } k \text{ used edge } (i, j), \text{ otherwise } 0$$

The Ant Colony System changes three things [14]. With a probability q_0 an ant takes the best edge, $j = \arg \max_l \tau_{il} \eta_{il}^\beta$, and otherwise it uses the probabilities above. Every ant that crosses an edge reduces its pheromone at once, $\tau_{ij} \leftarrow (1 - \xi) \tau_{ij} + \xi \tau_0$, so that later ants try other edges. Only the best tour found so far deposits pheromone, $\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \rho / L_{\text{best}}$.

Worked example: Choosing holes and updating pheromone

A drill must visit four holes at the corners of a rectangle: A (0, 0), B (3, 0), C (3, 4) and D (0, 4), in centimetres. The distances are $d_{AB}=d_{CD}=3$, $d_{AD}=d_{BC}=4$ and $d_{AC}=d_{BD}=\sqrt{3^2+4^2}=5$. The parameters are $\alpha=1$, $\beta=2$, $\rho=0.3$ and $Q=1$, and all pheromone values start at 1.

Step 1. The first decision of an ant at A. With $\tau=1$, only the heuristic matters: $\eta_{AB}^2=(1/3)^2=0.1111$, $\eta_{AC}^2=(1/5)^2=0.04$ and $\eta_{AD}^2=(1/4)^2=0.0625$, which add up to 0.2136. Hence

$$p_{AB}=\frac{0.1111}{0.2136}=0.5201, p_{AC}=\frac{0.04}{0.2136}=0.1873, p_{AD}=\frac{0.0625}{0.2136}=0.2926$$

Step 2. Two tours. Ant 1 builds A, B, C, D, A with length $3+4+3+4=14$, and ant 2 builds A, C, B, D, A with length $5+4+5+4=18$. Their deposits are $1/14=0.071429$ and $1/18=0.055556$.

Step 3. Evaporation and deposit. Every edge keeps $(1-0.3) \times 1=0.7$ and receives the deposits of the ants that used it:

Edge	Length	New pheromone τ
AB	3	$0.7 + 0.071429 = 0.771429$
AC	5	$0.7 + 0.055556 = 0.755556$
AD	4	$0.7 + 0.071429 + 0.055556 = 0.826985$
BC	4	$0.7 + 0.071429 + 0.055556 = 0.826985$
BD	5	$0.7 + 0.055556 = 0.755556$
CD	3	$0.7 + 0.071429 = 0.771429$

Step 4. The next decision at A. The products $\tau \eta^2$ are $0.771429 \times 0.1111=0.085706$ for B, $0.755556 \times 0.04=0.030222$ for C and $0.826985 \times 0.0625=0.051687$ for D, with the sum 0.167615, so

$$p_{AB}=0.5113, p_{AC}=0.1803, p_{AD}=0.3084$$

Edge AD, which belongs to both tours, gained weight, and the diagonal AC, which belongs only to the longer tour, lost weight. Over many iterations this feedback concentrates the pheromone on the short tour around the rectangle, of length 14. Part E of the [interactive lab](#) repeats these calculations for any parameters and tours.

Artificial bee colony and the metaphor question

Karaboga and Basturk's artificial bee colony algorithm assigns roles to solutions [5]. Employed bees search around their food sources, onlooker bees choose sources in proportion to their quality and search around them, and scout bees replace sources that have not improved for a number of trials with random new ones. The scout phase gives the algorithm a built-in restart mechanism. Many further algorithms followed, inspired by fireflies, bats, wolves, whales and others, including the ant lion optimiser used to train neural networks for chaotic signal prediction in earlier work [16].

Sörensen argued that the flood of metaphor-based algorithms often hides old ideas behind new vocabulary and weak experiments [6]. The no free lunch theorems add that no algorithm can be best on all problems [17]. Engineers should therefore judge a metaheuristic by its search mechanism rather than its metaphor, compare it with established methods on the problem at hand, report results over many independent runs with statistics, and count function evaluations, not iterations, because a single evaluation of an engineering simulation may take minutes.

Check your understanding. A paper claims that a new animal-inspired algorithm beats PSO, based on one run per method with different numbers of function evaluations. What is the main weakness?

- A. The metaphor is not biological enough
- B. One run and unequal evaluation budgets do not support a fair comparison of stochastic algorithms
- C. PSO cannot be compared with other algorithms
- D. The paper should have used a larger swarm only for the new algorithm

Answer: B. Fair comparisons need equal budgets, many independent runs and statistical summaries.

Formulas: The artificial bee colony

The colony keeps SN food sources $x_1, \dots, x_{SN}$, each a candidate solution [5]. An employed bee changes one randomly chosen component j of its source i , using a randomly chosen other source k and a random number ϕ between -1 and 1 :

$$v_{ij} = x_{ij} + \phi(x_{ij} - x_{kj})$$

and keeps the new source only if it is better. For a minimisation problem with objective values f_i , the fitness of a source is

$$\text{fit}_i = \frac{1}{1+f_i} \text{ if } f_i \geq 0, \text{fit}_i = 1 + |f_i| \text{ if } f_i < 0$$

Onlooker bees choose sources with the probabilities

$$p_i = \frac{\text{fit}_i}{\sum_{n=1}^{SN} \text{fit}_n}$$

and search around them in the same way. Every source counts the trials that failed to improve it; a source whose counter exceeds a limit is abandoned, and a scout bee replaces it with a random source $x_{ij} = l_j + \text{rand}(0, 1)(u_j - l_j)$ [5].

Worked example: Employed bees, onlookers and the scout

The objective is $f(x) = x_1^2 + x_2^2$ with three sources: $x_1 = (1, 2)$ with $f = 1 + 4 = 5$, $x_2 = (-1, 0.5)$ with $f = 1 + 0.25 = 1.25$, and $x_3 = (0.5, -1.5)$ with $f = 0.25 + 2.25 = 2.5$.

Step 1. An employed bee at source 1. It changes the second component, $j = 2$, with the partner $k = 2$ and $\phi = -0.4$:

$$v_{12} = 2 + (-0.4) \times (2 - 0.5) = 2 - 0.6 = 1.4$$

The new source $(1, 1.4)$ has $f = 1 + 1.4^2 = 2.96$, better than 5, so it replaces source 1 and its trial counter returns to 0. Sources 2 and 3 are assumed not to improve in this phase, so their counters rise by one.

Step 2. Fitness and probabilities.

$$\text{fit}_1 = \frac{1}{1+2.96} = 0.252525, \text{fit}_2 = \frac{1}{1+1.25} = 0.444444, \text{fit}_3 = \frac{1}{1+2.5} = 0.285714$$

$$p_1 = \frac{0.252525}{0.982683} = 0.257, p_2 = \frac{0.444444}{0.982683} = 0.4523, p_3 = \frac{0.285714}{0.982683} = 0.2907$$

where $0.982683 = 0.252525 + 0.444444 + 0.285714$.

Step 3. An onlooker. The cumulative probabilities are 0.257, 0.7093 and 1. A random number of 0.6 falls between 0.257 and 0.7093, so the onlooker searches around source 2, the best one, exactly as an employed bee would.

Step 4. The scout. If source 3 failed to improve in, say, 20 consecutive trials while the limit is 20, a scout replaces it with a random point of the search region. This restart keeps the colony from stagnating, which is the mechanism behind the metaphor. Part G of the [interactive lab](#) runs all three phases with every number shown.

Python step 6: Classes, objects and probabilistic choice

Why classes

Simulation code keeps state that changes over time, such as the speed and the current of a motor, together with the functions that change it. A class bundles both: Attributes store the state, and methods are functions that act on it. The class is the blueprint, and every object created from it has its own attributes. The special method `__init__` runs when an object is created and sets its initial state, and the first parameter of every method, `self`, refers to the object itself.

The DC motor of this week has the parameters of the speed example of the Control Tutorials for MATLAB and Simulink [13]. Its speed w and current i obey $J \frac{dw}{dt} = K i - b w$ and $L \frac{di}{dt} = V - R i - K w$, which the method `step` integrates over small time steps.

```
class DCMotor:
    """Armature-controlled DC motor with the parameters of the CTMS
    speed example."""

    def __init__(self, J=0.01, b=0.1, K=0.01, R=1.0, L=0.5):
        self.J, self.b, self.K, self.R, self.L = J, b, K, R, L
        self.speed = 0.0          # rad/s
        self.current = 0.0        # A

    def step(self, voltage, dt=0.001):
        """Advance the motor by dt seconds with the explicit Euler
        method."""
        acc = (self.K * self.current - self.b * self.speed) / self.J
        di = (voltage - self.R * self.current - self.K * self.speed) /
self.L
        self.speed += acc * dt
        self.current += di * dt
        return self.speed

motor = DCMotor()
for _ in range(3000):          # three seconds in steps of 1 ms
    motor.step(1.0)
print(f"speed after 3 s at 1 V: {motor.speed:.4f} rad/s")
```

Output

speed after 3 s at 1 V: 0.0996 rad/s

The underscore `_` is the conventional name for a loop variable that is not used. The speed approaches $K V / (b R + K^2)$, about 0.1 rad/s for 1 V, which the steady-state analysis of the tutorial also gives.

Many objects from one class

Objects created from the same class are independent. A motor with a lighter rotor does not share its speed with a motor with a heavier one, and each object keeps its own attributes. This independence makes classes practical for swarms, fleets of vehicles and banks of sensors.

```
light = DCMotor(J=0.005)
heavy = DCMotor(J=0.05)
```

```

for _ in range(200):          # 0.2 s
    light.step(1.0)
    heavy.step(1.0)
print(f"light rotor: {light.speed:.4f} rad/s, heavy rotor:
{heavy.speed:.4f} rad/s")

```

Output

light rotor: 0.0257 rad/s, heavy rotor: 0.0061 rad/s

A particle of the swarm

Particle swarm optimisation moves each particle with a velocity that combines its previous velocity, a pull towards its own best position and a pull towards the best position of the swarm. A class keeps the position, the velocity and the personal best of a particle together, and NumPy arrays let the same code work in any number of dimensions. The method `copy` creates an independent array: Without it, the personal best and the current position would be two names for the same array, and changing one in place would also change the other.

```

import numpy as np

class Particle:
    """One particle of a swarm with position, velocity and personal
    best."""

    def __init__(self, position, velocity):
        self.x = np.array(position, dtype=float)
        self.v = np.array(velocity, dtype=float)
        self.best_x = self.x.copy()

    def move(self, swarm_best, w=0.7, c1=1.5, c2=1.5, r1=0.5, r2=0.5):
        """Update velocity and position; r1 and r2 stand in for random
        numbers."""
        self.v = w * self.v + c1 * r1 * (self.best_x - self.x) + c2 *
r2 * (swarm_best - self.x)
        self.x = self.x + self.v

p = Particle([2.0, -1.0], [0.1, 0.0])
p.move(swarm_best=np.array([0.0, 0.0]))
print(p.x, p.v)

```

Output

[0.57 -0.25] [-1.43 0.75]

The notebook of this week builds a complete swarm on the same pattern, with new random numbers at every step.

Probabilistic choice in an ant colony

Ant colony optimisation builds solutions step by step [3]. At every step an ant chooses among the possible next moves with probabilities proportional to $\tau^\alpha \eta^\beta$, where τ is the pheromone on a move and η its heuristic desirability, usually one over the distance. The generator method `choice` draws from such a discrete distribution when the probabilities are passed as `p`, and `np.bincount` counts how often each move was chosen.

```

tau = np.ones(4) # pheromone on four
candidate edges
dist = np.array([4.0, 2.0, 5.0, 3.0]) # edge lengths in km
alpha, beta = 1.0, 2.0
weight = tau ** alpha * (1 / dist) ** beta
prob = weight / weight.sum()
print(prob.round(3))
colony = np.random.default_rng(5)
choices = colony.choice(4, size=1000, p=prob) # the choices of 1000
ants
print(np.bincount(choices, minlength=4))

```

Output

```

[0.135 0.539 0.086 0.24 ]
[148 532  78 242]

```

After the ants have moved, pheromone evaporates at a rate ρ , and every ant deposits pheromone on its choice, more on shorter edges. Repeated over many iterations, this positive feedback concentrates the colony on short routes, while evaporation keeps it from settling too early on a poor one.

```

rho = 0.5
deposit = np.bincount(choices, minlength=4) / dist / len(choices) #
1/length per ant, scaled by the colony size
tau = (1 - rho) * tau + deposit
print(tau.round(3))

```

Output

```

[0.537 0.766 0.516 0.581]

```

Check your understanding. A class Counter sets `self.n = 0` in `init` and has a method `add` that runs `self.n += 1`. After `a = Counter()`, `b = Counter()`, `a.add()`, `a.add()` and `b.add()`, what are `a.n` and `b.n`?

- A. 2 and 1
- B. 3 and 3
- C. 1 and 1
- D. 2 and 2

Answer: A. Each object keeps its own attributes, so the calls on `a` do not change `b`.

Discipline challenges

Swarms tune continuous parameters and ants build sequences. Choose a row, write the cost function, and compare the swarm with a simple baseline.

Department	Challenge
Electrical and Electronics Engineering	PID tuning of a DC motor or of a buck converter output voltage loop, as in the notebook.
Mechanical Engineering and Automotive Engineering	Suspension damping and stiffness that trade comfort against road holding.
Industrial Engineering	Vehicle routing for deliveries from a depot with ant colony optimisation.
Computer Engineering	Routing in a communication network or task allocation among cores with ants.
Civil Engineering	Size optimisation of a planar truss with stress constraints using PSO.

Department	Challenge
Environmental Engineering	Calibration of a river water-quality model to measured oxygen profiles.
Mining Engineering	Sequencing of blast holes or of stope extraction with ants.
Geophysical Engineering	Inversion of a layered-earth resistivity model from sounding curves with PSO.
Chemical Engineering and Chemistry	Estimation of kinetic parameters of a reaction from concentration curves.
Textile Engineering	Cutting-order planning and marker layout sequencing.
Biology and Statistics	Parameter estimation of a population growth model, compared with least squares.

Weekly task

Apply particle swarm optimisation or ant colony optimisation to a problem from your department. For a continuous problem, compare PSO with differential evolution from Week 5 under the same evaluation budget over ten seeds. For a sequencing problem, compare the ant colony with the nearest-neighbour heuristic. Report the setup, a convergence plot and a statistical summary in about 500 words, citing at least three works from this week's references [2, 3, 6].

Research and report assignment (optional)

Swarm intelligence in engineering practice. Review applications of particle swarm, ant colony or bee colony algorithms in one field, such as power systems, structural design, logistics or controller tuning. Assess the quality of the experimental comparisons in the reviewed papers against the criteria of Sörensen and against the no free lunch theorems [6, 12, 17].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Bonabeau, E., Dorigo, M., & Theraulaz, G. (1999). *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press.
- [2] Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95, International Conference on Neural Networks*, Vol. 4 (pp. 1942-1948). IEEE.
<https://doi.org/10.1109/ICNN.1995.488968>
- [3] Dorigo, M., Maniezzo, V., & Colorni, A. (1996). Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1), 29-41.
<https://doi.org/10.1109/3477.484436>
- [4] Dorigo, M., & Stützle, T. (2004). *Ant Colony Optimization*. MIT Press.
- [5] Karaboga, D., & Basturk, B. (2007). A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *Journal of Global Optimization*, 39(3), 459-471.
<https://doi.org/10.1007/s10898-007-9149-x>
- [6] Sörensen, K. (2015). Metaheuristics: The metaphor exposed. *International Transactions in Operational Research*, 22(1), 3-18. <https://doi.org/10.1111/itor.12001>

- [7] Python Software Foundation (2026). *The Python Tutorial (Python 3 documentation)*.
<https://docs.python.org/3/tutorial/>
- [8] Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. *ACM SIGGRAPH Computer Graphics*, 21(4), 25-34. <https://doi.org/10.1145/37402.37406>
- [9] Shi, Y., & Eberhart, R. (1998). A modified particle swarm optimizer. In *1998 IEEE International Conference on Evolutionary Computation Proceedings* (pp. 69-73). IEEE.
<https://doi.org/10.1109/ICEC.1998.699146>
- [10] Clerc, M., & Kennedy, J. (2002). The particle swarm: Explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1), 58-73.
<https://doi.org/10.1109/4235.985692>
- [11] Åström, K. J., & Hägglund, T. (1995). *PID Controllers: Theory, Design, and Tuning* (2nd ed.). Instrument Society of America.
- [12] Gaing, Z.-L. (2004). A particle swarm optimization approach for optimum design of PID controller in AVR system. *IEEE Transactions on Energy Conversion*, 19(2), 384-391.
<https://doi.org/10.1109/TEC.2003.821821>
- [13] University of Michigan, Carnegie Mellon University, & University of Detroit Mercy (2026). *Control Tutorials for MATLAB and Simulink: DC motor speed, system modeling*. <https://ctms.engin.umich.edu>
- [14] Dorigo, M., & Gambardella, L. M. (1997). Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1), 53-66.
<https://doi.org/10.1109/4235.585892>
- [15] Reinelt, G. (1991). TSPLIB: A traveling salesman problem library. *ORSA Journal on Computing*, 3(4), 376-384. <https://doi.org/10.1287/ijoc.3.4.376>
- [16] Kose, U. (2018). An ant-lion optimizer-trained artificial neural network system for chaotic electroencephalogram (EEG) prediction. *Applied Sciences*, 8(9), 1613. <https://doi.org/10.3390/app8091613>
- [17] Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67-82. <https://doi.org/10.1109/4235.585893>